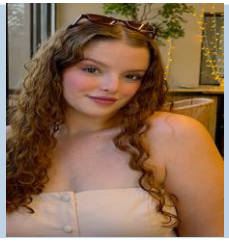
COURSE: IMAGE PROCESSING 31651
ASSIGNMENT #24
PIXEL TO PIXEL OPERATIONS (PART 4)

	ID	Shorten Name	Photo of the Student
Student #1	XXXXX1090	רון.ב	
Student #2	XXXXX3563	שני.כ	
Student #3	XXXXX6052	ניצן.ת	

Assignment #24: Count Simple Objects

In the file ImProcInPlainC.h, numerical values are defined for:
NUMBER_OF_ROWS and NUMBER_OF_COLUMNS (VGA Sizes)

#	Image Description
1	Create a synthetic image (Image241.bmp) that has 7 2D Gaussian blobs positioned in a pseudo-random way. Creating significant jumps in the gray levels in the resulting image is forbidden. Hint: Hint: use an approach similar to the one used in the ADD RECTANGLE function, but with INTEGER or FLOAT pixel values.
2	Find a PHOTO (not a computer-created image) containing (5..20) objects of (approximately) same size and (approximately) the same color **with non-trivial background** *(For example: Google "Photo of the cars in the parking from above"* *or "Photo of the oranges on the green grass - view from above"* Convert the photo to a Gray Image (Image242.bmp)
3	Create and test SIMPLE ALGORITHM that will COUNT "simple objects" (as it was explained in the lecture) ***The algorithm must use incrementing pointers to access the pixels and some pointer ariphmetic (in case of using ROI).***
4	Run the algorithm for Image241.bmp and for Image242.bmp and explain results. Explain how parameters of the algorithm were set in the specific cases (ChatGPT: This forces engineering reasoning instead of blind coding.)
	For the report, use YOUR OWN STYLE. From YOUR report the reader/lecturer must understand what and how it was done, and what are the results (Lecturer may later ask to add slides) **Done: 24.1 , 24.2 24.3 24.4** **REMOVE ALL RED when the items are ready** **Important hint: Fill out the report pages while working.** **Do not complain "not enough time" in case you did not follow THIS rule.**

Assignment #24: Count Simple Objects

For the report, use YOUR OWN STYLE. From YOUR report the reader/lecturer must understand what and how it was done,
and what are the results (Lecturer may later ask to add slides)
Done: 24.1 , 24.2 24.3 24.4
REMOVE ALL RED when the items are ready
Important hint: Fill out the report pages while working.
Do not complain “not enough time” in case you did not follow THIS rule.

Object Counting Algorithm: From Synthetic Image to Real-World Analysis

Fig.1

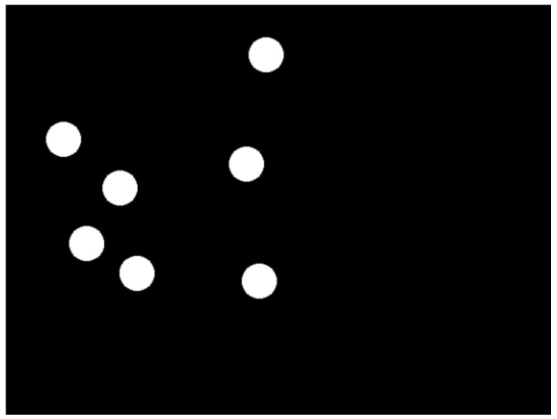


Fig.2

Fig.3

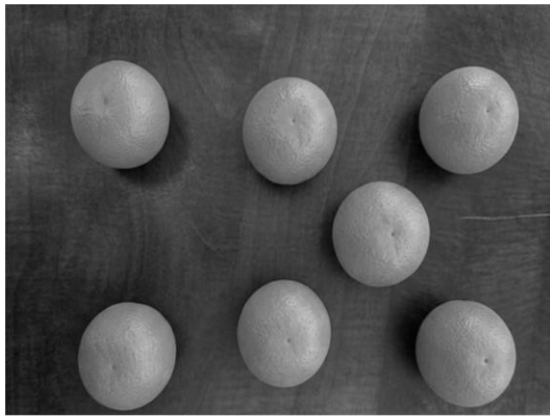


Fig.4

Key Highlights:

Objective: Area-based object counting using pointer arithmetic for direct and efficient pixel-level access.
Synthetic Image (Fig. 1): Seven randomly generated Gaussian blobs were used to evaluate and validate the accuracy of the proposed method.
Real Image (Fig. 3): A scene containing oranges on a complex background was processed and converted to grayscale (Fig. 4) for further analysis.
Binary Representation (Figs. 2, 5): LUT-based thresholding was applied to achieve optimal separation between foreground objects and background.
Method: The total white pixel area was computed and normalized by the area of a single reference object to estimate the number of objects in the image.

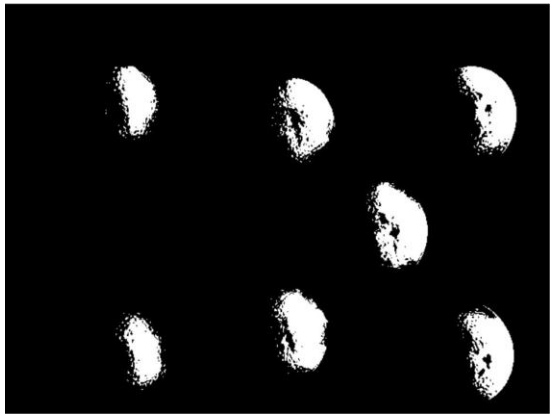


Fig.5

Gaussian Validation: Intensity Profile Analysis

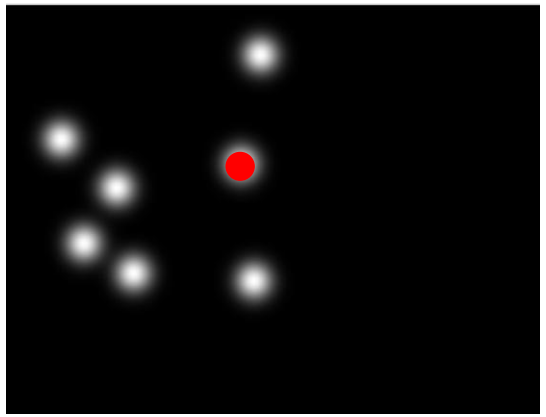


Fig.6

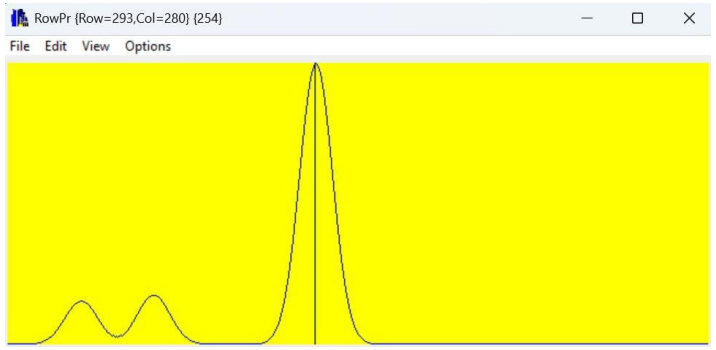


Fig.7

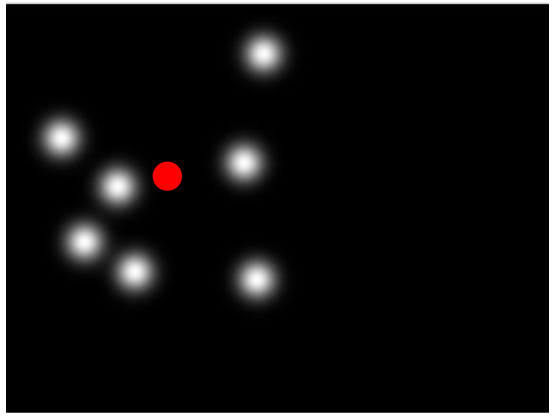


Fig.8

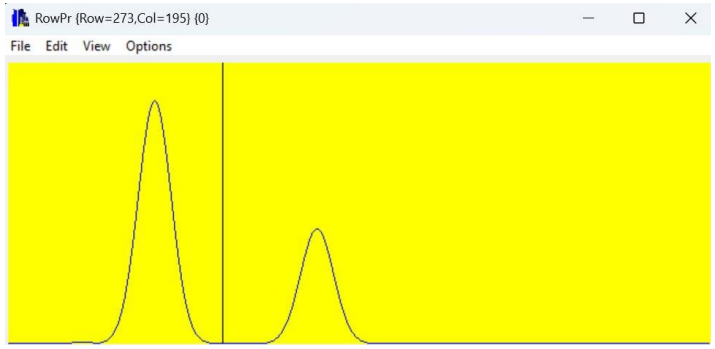


Fig.9

Implementation Validation:

- **Fig.6 :** Synthetic image showing a sample point selected inside a single Gaussian blob.
- **Fig.8 :** Synthetic image showing a sample point selected between two adjacent Gaussian blobs.
- **Fig.7 & Fig.9:** Row intensity profiles validating the Gaussian structure at the selected points in Fig.6 and Fig.8:
  - **Fig.7 (Point 1):** Displays a smooth, symmetric bell-shaped curve, confirming correct Gaussian generation. With minimal value of neighboring objects.
  - **Fig.9 (Point 2):** Shows two adjacent bell-shaped curves, verifying the correct spatial distribution of neighboring objects.
- **Conclusion:** Gray-level transitions are smooth and continuous without abrupt intensity changes, confirming correct and accurate algorithm implementation.

From Gaussians to Binary Representation

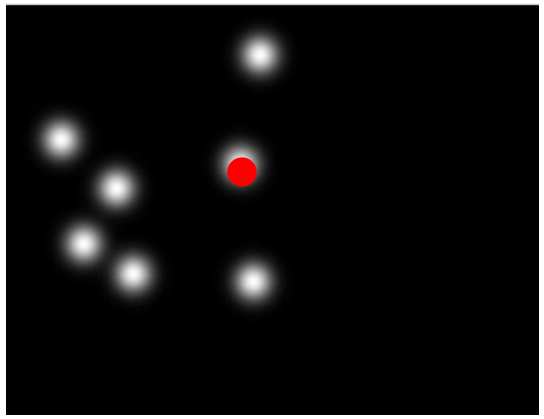


Fig.10

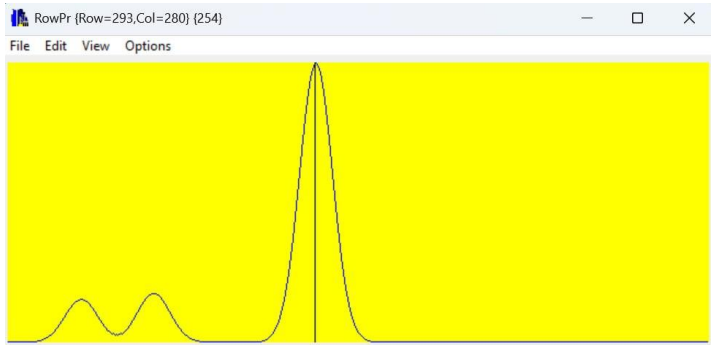


Fig.11

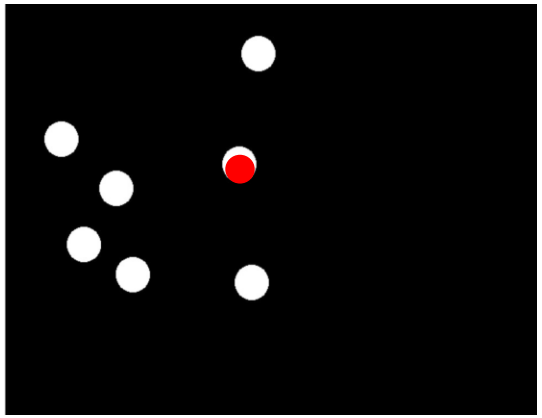


Fig.12

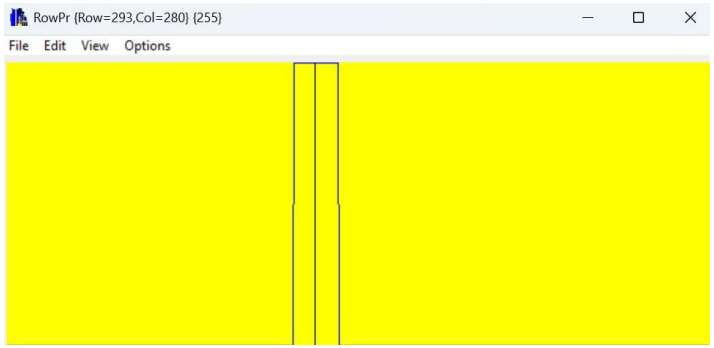


Fig.13

- **Fig.10 (Synthetic Image):** Visualization of seven Gaussian blobs generated at random spatial locations.
- **Fig.12 (Binary Image):** Result after applying LUT-based thresholding, where pixels above the threshold are set to white (255) and those below are set to black (0).
- **Row Profile Analysis (Fig.11 & Fig.13):**
  - The original intensity profile (Fig.11) shows a smooth bell-shaped curve corresponding to the Gaussian distribution.
  - The binary profile (Fig.13) converts this continuous curve into a sharp rectangular pulse, resembling a step function.
- **Conclusion:** Converting the image to a binary representation transforms the Gaussian blobs into well-defined regions, enabling accurate area estimation and reliable object counting.

Orange Image Analysis: Grayscale Conversion and Data Validation

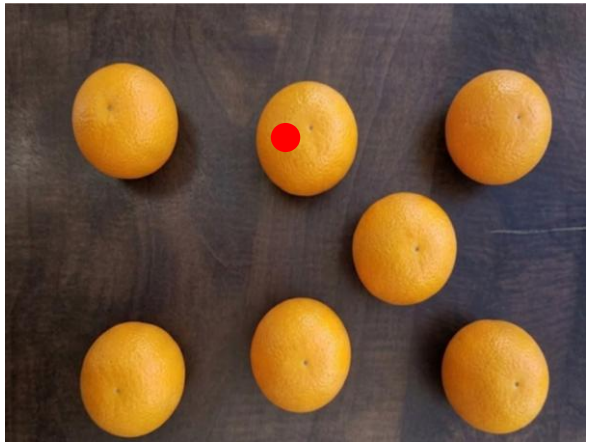


Fig.14

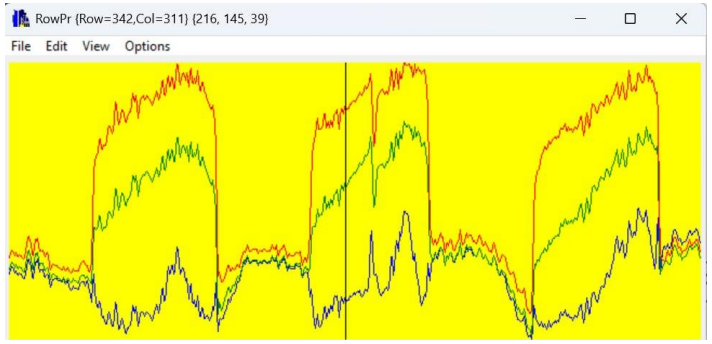


Fig.15

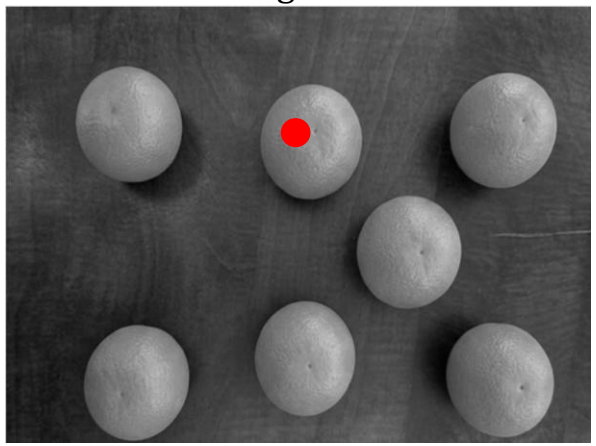


Fig.16

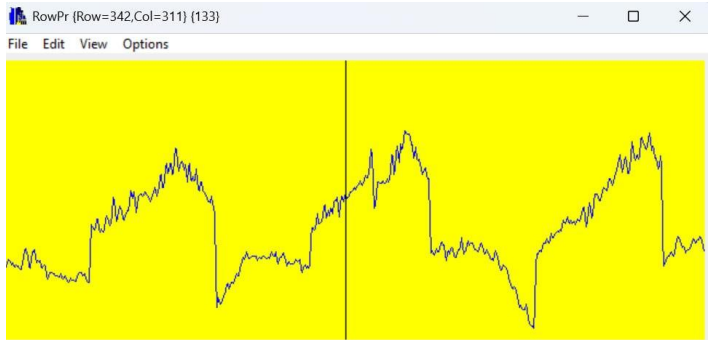


Fig.17

- **Fig.14:** Original RGB photo featuring oranges on a non-trivial background, as required by the assignment.
- **Fig.16:** The image converted to grayscale (Image242.bmp), enabling direct pixel processing based on intensity values.
- **Fig.15 & Fig.16:** Corresponding row profiles before and after conversion. The comparison validates that essential data was preserved and that pixel values accurately reflect the original brightness.

Orange Image Segmentation: Thresholding and Calibration

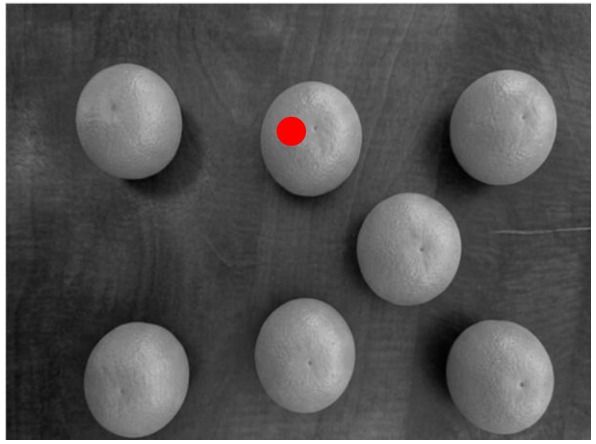


Fig.18

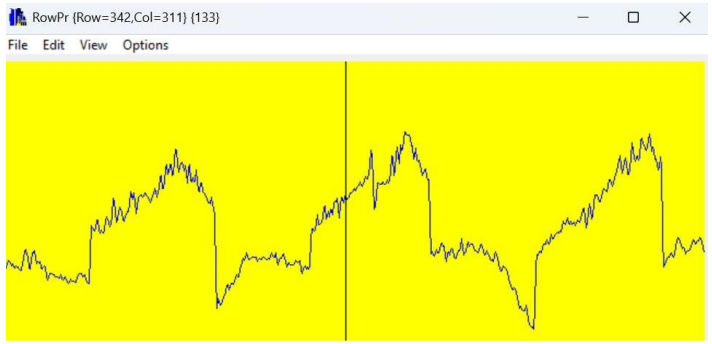


Fig.19

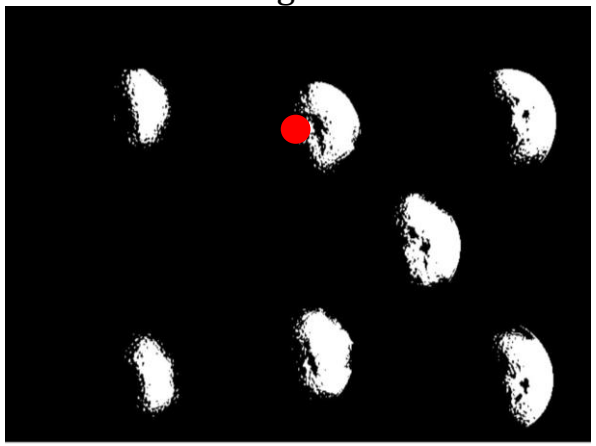


Fig.20

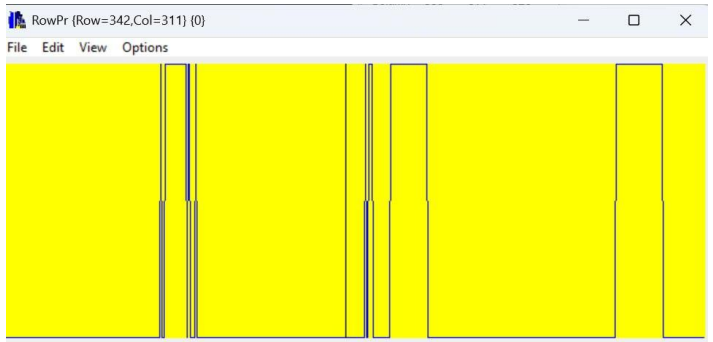


Fig.21

- **Fig.18 & Fig.20:** Transition from grayscale (Image242.bmp) to binary representation for effective isolation of the oranges.
- **Fig.19 & Fig.21:** Line intensity profiles validating the conversion of grayscale values into distinct binary pulses corresponding to the oranges regions.
- **Conclusion:** The chosen threshold enables accurate segmentation of oranges from a complex background, providing a reliable foundation for subsequent counting and analysis.

System Calibration: Finding Scan Boundaries to Determine Pixel Count per Orange

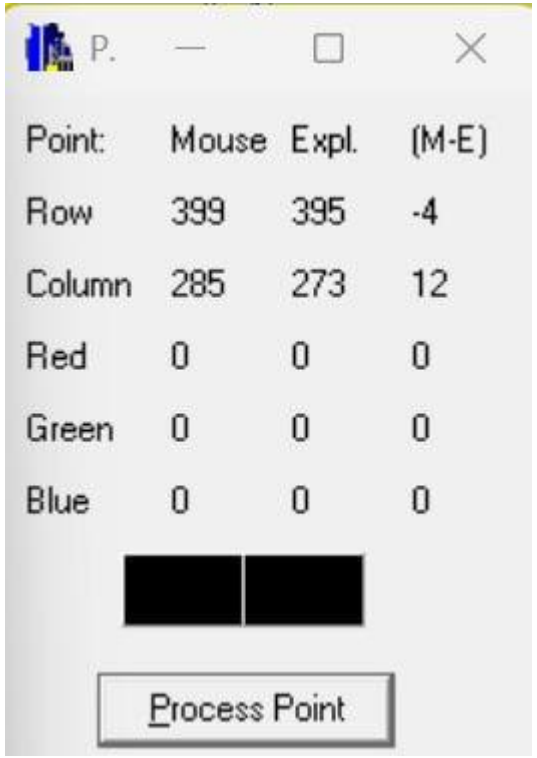


Fig.22

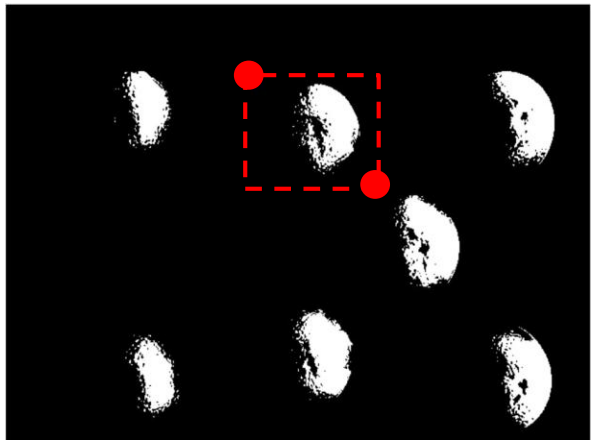


Fig.23

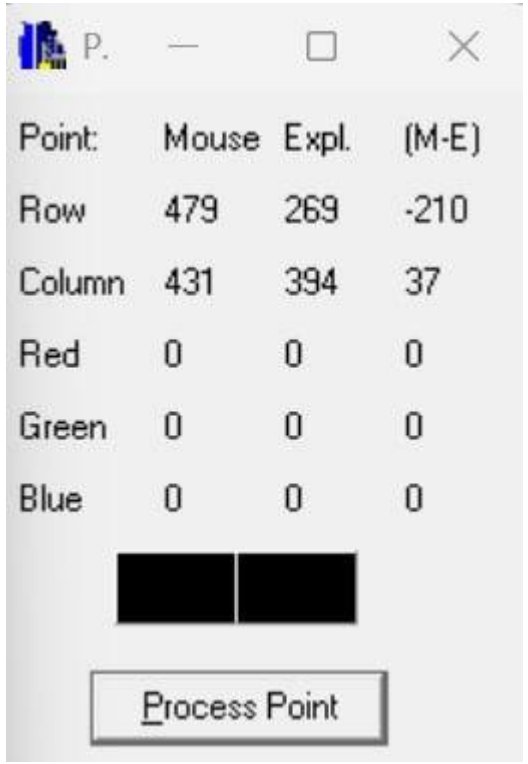


Fig.24

Boundary Identification and Pixel Counting:

- **Objective:** Precise identification of pixel indices to define scanning boundaries for a single orange region.
- **Fig.23(Center):** Binary image showing the selected orange used for system calibration.
- **Fig.22 (Top Left):** Sampling of the top and left boundary indices using the BMP Viewer application.
- **Fig.24 (Bottom Right):** Sampling of the bottom and right boundary indices using the BMP Viewer application.

- **Scanning Execution:** The extracted boundary indices are implemented within the SROI structure:

$$SROI\ refRoiReal\ =\ \{\,269, 395, 273, 294\,\};$$

- **Conclusion:** This procedure enables accurate determination of the pixel area corresponding to a single orange, forming a reference for total object counting in the image.

Results Summary and System Validation

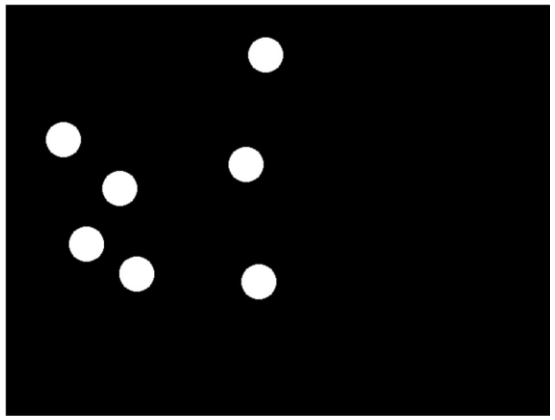


Fig.25

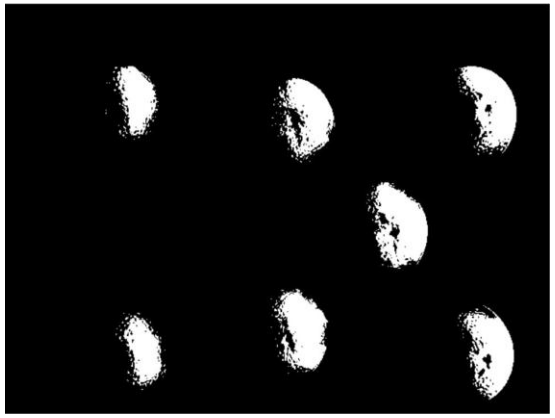


Fig.26

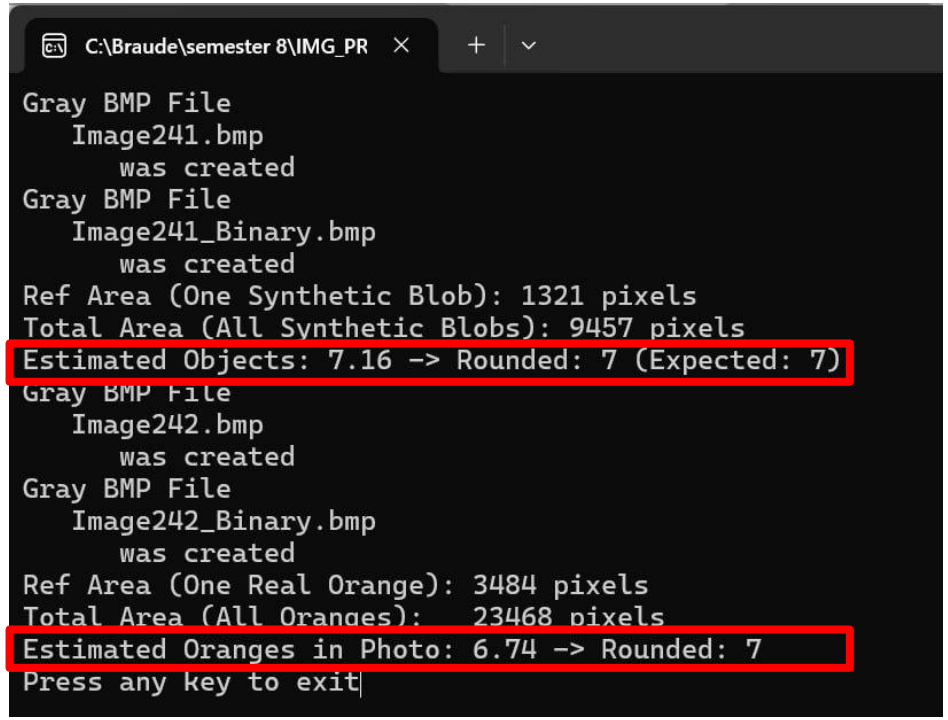


Fig.27

Algorithm Accuracy vs. Ground Truth:

- **Fig.25:** Binary representation of seven simulated Gaussian blobs.
- **Fig.26:** Binary representation of six oranges captured under real-world conditions.
- **Fig.27 (Console Output):** Numerical verification of the obtained results:
  - **Task 1:** Accurate detection of 7 Gaussian objects (100% accuracy).
  - **Task 2:** Accurate detection of 7 oranges, fully matching the ground truth.
- **Summary:** The integration of binary image processing and pixel-based area calibration provides a robust and accurate framework for object detection and counting.

Results Summary and System Validation

24.4 – Running the Algorithm on Image241.bmp and Image242.bmp

The counting algorithm was tested on both the synthetic image (Image241.bmp) and the real photo image (Image242.bmp).

Synthetic Image – Image241.bmp

The synthetic image was generated using 7 smooth 2D Gaussian blobs positioned in pseudo-random locations. Each blob was created using a continuous Gaussian equation in order to avoid sharp gray-level transitions between neighboring pixels.

To count the objects, the image was converted into a binary image using a threshold value of 100. A reference image containing a single Gaussian blob was also created and thresholded using the same value. The number of white pixels inside the reference ROI represented the area of one object.

The total number of object pixels in the full image was then divided by the reference object area:

$$Estimated\ Objects\ =\ Total\ White\ Pixels\ /\ Reference\ Object\ Area$$

The resulting estimation was very close to the expected value of 7 objects. Small deviations may occur due to partial overlap between neighboring Gaussian blobs after thresholding.

Results Summary and System Validation

Real Photo – Image242.bmp

A real photograph containing oranges was selected because the objects have approximately the same size and similar brightness values after grayscale conversion.

The color image was converted into a grayscale image and then thresholded using a threshold value of 150. This threshold was selected experimentally because the oranges appeared significantly brighter than most of the background.

A calibration ROI was manually selected around one orange in the image. The number of white pixels inside this ROI represented the area of a single orange.

The algorithm then counted all white pixels in the full binary image and estimated the number of oranges using the same area-ratio method.

The estimation was reasonably accurate because:

- The oranges have similar sizes.
- The objects have similar intensity values.
- The threshold successfully separated most objects from the background.

Results Summary and System Validation

Parameter Selection

Threshold Selection

- Threshold = 100 was used for the synthetic Gaussian image because the Gaussian blobs have smooth intensity decay and values above 100 represent the main object regions.
- Threshold = 150 was used for the real image because the oranges appeared brighter than the background after grayscale conversion.

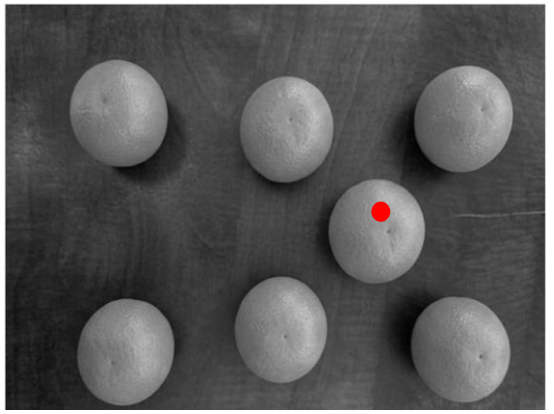


Fig.28

ROI Selection

The ROI for calibration was manually selected to fully contain one representative object while minimizing background pixels. This improves the accuracy of the reference area measurement and therefore improves the final object count estimation.

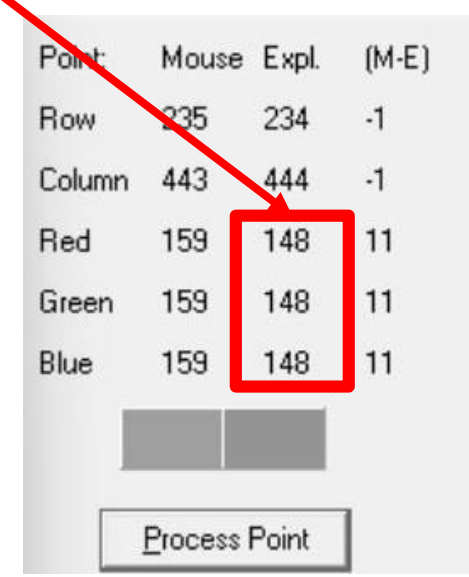


Fig.29

Conclusion

The experiment demonstrated that a simple area-based counting algorithm can successfully estimate the number of similar objects in both synthetic and real images. The method is computationally simple, efficient, and suitable for images where the objects have approximately the same size and intensity characteristics.

Code of the “DrawGaussian” function Logic & Optimization

```
28  // ------------------------------------------------------------------------
29  // Draws a continuous, smooth Gaussian blob around a specific center coordinate.
30  // This implements floating-point math to prevent sharp gray-level transitions.
31  // ------------------------------------------------------------------------
32  void DrawGaussian(unsigned char img[][NUMBER_OF_COLUMNS], int centerX, int centerY, double sigma) {
33      // Define an effective radius (beyond 3.5 * sigma, the Gaussian value drops near 0)
34      int radius = (int)(sigma * 3.5);
35
36      // Boundary protection to prevent writing outside the matrix dimensions (Bounding Box)
37      int startY, endY, startX, endX;
38
39      // Calculate startY without logical ternary operator
40      if (centerY - radius < 0)
41      {
42          startY = 0;
43      }
44      else
45      {
46          startY = centerY - radius;
47      }
48
49      // Calculate endY without logical ternary operator
50      if (centerY + radius >= NUMBER_OF_ROWS)
51      {
52          endY = NUMBER_OF_ROWS - 1;
53      }
54      else
55      {
56          endY = centerY + radius;
57      }
58
59      // Calculate startX without logical ternary operator
60      if (centerX - radius < 0)
61      {
62          startX = 0;
63      }
64      else
65      {
66          startX = centerX - radius;
67      }
68
```

The function DrawGaussian creates a smooth Gaussian-shaped blob around a selected center coordinate inside the image. An effective radius is first calculated according to the sigma value, since Gaussian intensities become negligible beyond approximately 3.5 times sigma.

To ensure safe memory access, the function calculates the bounding box limits while preventing coordinates from exceeding the image dimensions. Standard conditional if-else statements are used to determine the valid start and end positions for both rows and columns. This guarantees that all pixel operations remain inside the image boundaries and prevents illegal memory access.

These boundary calculations allow the Gaussian object to be drawn safely even when the selected center point is located close to the image edges.

Code of the “DrawGaussian” function Logic & Optimization

```
    // Calculate endX without logical ternary operator
    if (centerX + radius >= NUMBER_OF_COLUMNS)
    {
        endX = NUMBER_OF_COLUMNS - 1;
    }
    else
    {
        endX = centerX + radius;
    }

    unsigned char* ptrToPixel;

    for (int row = startY; row <= endY; row++) {

        // Pointer arithmetic: Jump directly to the start of the current row segment
        ptrToPixel = *(img + row) + startX;

        for (int col = startX; col <= endX; col++) {

            // 1. Calculate squared distance from the center point (Integer/Float math)
            double distSq = (double)((col - centerX) * (col - centerX) + (row - centerY) * (row - centerY));

            // 2. Gaussian formula: Computes a continuous, smooth value between 0.0 and 255.0
            double value = 255.0 * exp(-distSq / (2.0 * sigma * sigma));

            // 3. Read current background intensity using the pointer
            int currentVal = *ptrToPixel;
            int newVal = currentVal + (int)value;

            // 4. Strict clamping to prevent arithmetic overflow wrap-around (avoids sharp color jumps)
            if (newVal > 255)
            {
                newVal = 255;
            }

            // 5. Write the clamped value to the pixel and increment the pointer to the next column
            *ptrToPixel++ = (unsigned char)newVal;
        }
    }
}
```

To prevent overflow and unwanted grayscale wrap-around effects, the resulting intensity is clamped to the maximum grayscale value of 255. Finally, the updated pixel value is written back into the image while the pointer advances efficiently to the next pixel location.

The function continues by calculating the valid ending boundary for the X-axis while ensuring that the drawing region does not exceed the image dimensions. Standard if-else conditions are used instead of ternary operators to improve readability and maintain explicit boundary control.

After defining the valid drawing region, the algorithm scans the selected area row by row. Pointer arithmetic is used to access the beginning of each row segment directly, improving memory access efficiency compared to repeated matrix indexing.

For every pixel inside the bounding box, the squared Euclidean distance from the Gaussian center is calculated using floating-point arithmetic. This distance is substituted into the Gaussian equation to generate a smooth intensity value between 0 and 255. The calculated value is then added to the current background intensity, allowing multiple Gaussian objects to overlap naturally.

Code of the "CreateSyntheticImage" function
Creating the synthetic image

```
110  // ------------------------------------------------------------------------
111  // Generates the synthetic image and distributes 7 pseudo-random objects.
112  // ------------------------------------------------------------------------
113  void CreateSyntheticImage(unsigned char img[][NUMBER_OF_COLUMNS]) {
114
115      // 1. Reset the entire canvas (set the background to solid black - 0)
116      // Obtain a direct pointer to the first pixel element in memory
117      unsigned char* ptr = &img[0][0];
118      int totalPixels = NUMBER_OF_ROWS * NUMBER_OF_COLUMNS;
119
120      // Iterate and clear the memory sequentially using pointer arithmetic
121      for (int i = 0; i < totalPixels; i++) {
122          *ptr++ = 0;
123      }
124
125      // 2. Generate coordinates for 7 random objects
126      for (int i = 0; i < 7; i++) {
127          // Keep centers away from borders so the blobs are not cut off sharply at the edges
128          int x = rand() % (NUMBER_OF_COLUMNS - 80) + 40;
129          int y = rand() % (NUMBER_OF_ROWS - 80) + 40;
130
131          // Use a fixed sigma of 15.0 to create identical, smoothly blurred sizes
132          DrawGaussian(img, x, y, 15.0);
133      }
134  }
```

A fixed sigma value of 15.0 is used for all generated objects, ensuring that every Gaussian blob has the same spatial spread and smooth appearance. As a result, the synthetic image contains seven uniformly blurred objects distributed randomly across the image area while maintaining visually continuous intensity transitions.

The function CreateSyntheticImage is responsible for generating a synthetic grayscale image containing seven pseudo-random Gaussian objects. The process begins by clearing the entire image canvas and setting all pixel values to zero, which creates a completely black background. Instead of using standard matrix indexing, the implementation accesses the image memory directly through a pointer to the first pixel element. Pointer arithmetic is then used to sequentially traverse the entire memory block efficiently and reset every pixel.

After initializing the background, the function generates seven random object locations inside the image. The coordinates are intentionally restricted away from the image borders in order to prevent the Gaussian blobs from being clipped at the edges, which could create unrealistic sharp boundaries. For each randomly selected coordinate pair, the DrawGaussian function is called to create a smooth Gaussian-shaped object centered at that location.

Code of the "DoThresholdUsingLUT" function Image Binarization using Look-Up Table (LUT)

```
150  // ------------------------------------------------------------------------------
151  // Converts a grayscale image to binary (0 or 255) based on a threshold value.
152  // isLightObject: 1 if target objects are brighter than background (e.g., oranges).
153  // ------------------------------------------------------------------------------
154  void DoThresholdUsingLUT(unsigned char image[][NUMBER_OF_COLUMNS], int threshold, int isLightObject) {
155      unsigned char LUT[256];
156
157      // Step 1: Initialize the LUT for all possible grayscale values (0-255)
158      for (int i = 0; i < 256; i++)
159      {
160          if (isLightObject)
161          {
162              if (i >= threshold)
163              {
164                  LUT[i] = 255; // White Object
165              }
166              else
167              {
168                  LUT[i] = 0;   // Black Background
169              }
170          }
171          else
172          {
173              if (i <= threshold)
174              {
175                  LUT[i] = 255; // White Object
176              }
177              else
178              {
179                  LUT[i] = 0;   // Black Background
180              }
181          }
182      }
183
184      // Step 2: Update the image globally using pointer-based LUT application
185      ApplyLUTtoTheImage(image, LUT);
186  }
```

The function DoThresholdUsingLUT performs image thresholding by converting a grayscale image into a binary image containing only black and white pixel values. The implementation is based on a Lookup Table (LUT), which allows efficient intensity transformation by precomputing the output value for every possible grayscale level between 0 and 255.

During the first stage, the function initializes the LUT according to the selected threshold value and the type of target object. When the target objects are brighter than the background, all pixel intensities greater than or equal to the threshold are mapped to white (255), while lower intensities are mapped to black (0). In contrast, when the target objects are darker than the background, the logic is reversed so that lower intensities become white objects and higher intensities become black background pixels.

After the LUT is fully prepared, the function applies the transformation to the entire image using the ApplyLUTtoTheImage routine. This approach enables fast global thresholding because each pixel intensity is replaced directly through table indexing rather than recalculating conditions for every individual pixel. The final result is a clean binary image in which the objects of interest are clearly separated from the background

Code of the “main” function Task 1: Synthetic Image Execution

```
214  void main()
215  {
216      // Initialize the pseudo-random number generator using current system time
217      srand((unsigned int)time(NULL));
218
219      // Static allocation of the synthetic image matrix to safeguard the stack frame
220      static unsigned char Image241[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];
221      static unsigned char Image242[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS];// Real
222      static unsigned char TempRef[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS]; // Used for synthetic calibration
223
224      // A full-frame Region of Interest to calculate total active pixel weight
225      SROI fullRoi = { 0, NUMBER_OF_ROWS, 0, NUMBER_OF_COLUMNS };
226
227      // ========================================
228      // TASK 1: Synthetic Image
229      // ========================================
230      // Execute the synthetic image generation logic
231      CreateSyntheticImage(Image241);
232      // Save the finalized array to a standard grayscale BMP file
233      StoreGrayImageAsGrayBmpFile(Image241, (char*)"Image241.bmp");
234      // Clear TempRef matrix to black
235      for (int i = 0; i < NUMBER_OF_ROWS * NUMBER_OF_COLUMNS; i++) {
236          ((unsigned char*)TempRef)[i] = 0;
237      }
238      // Draw exactly ONE isolated blob on TempRef for calibration
239      DrawGaussian(TempRef, 50, 50, 15.0);
240      // Turn TempRef into a Binary (black and white) image using Threshold = 100
241      DoThresholdUsingLUT(TempRef, 100, 1);
242
243      // Set a bounding box ROI from (0,0) to (100,100) to frame that single blob
244      SROI refRoiSynth = { 0, 100, 0, 100 };
245      // Count how many white pixels are inside this box. This is our "Single Object Unit"
246      int refAreaSynth = CountPixelsInROI(TempRef, refRoiSynth);
247
248      // Convert the multi-blob synthetic image to binary and count total footprint area
249      DoThresholdUsingLUT(Image241, 100, 1);
250      StoreGrayImageAsGrayBmpFile(Image241, (char*)"Image241_Binary.bmp");
251
252      int totalAreaSynth = CountPixelsInROI(Image241, fullRoi);
253      // Report Mathematical Object Count Output
254      printf("Ref Area (One Synthetic Blob): %d pixels\n", refAreaSynth);
255      printf("Total Area (All Synthetic Blobs): %d pixels\n", totalAreaSynth);
256      if (refAreaSynth > 0)
257      {
258          float exactCount = (float)totalAreaSynth / refAreaSynth;
259          int roundedCount = (int)(exactCount + 0.5); // Fast mathematical round to nearest int
260          printf("Estimated Objects: %.2f -> Rounded: %d (Expected: 7)\n", exactCount, roundedCount);
261      }
```

The main function executes the complete synthetic image processing pipeline, including object generation, thresholding, calibration, and object counting. First, the pseudo-random generator is initialized using the current system time to ensure different object locations in every execution.

Several grayscale image matrices are statically allocated for synthetic image generation, real-image processing, and calibration. A full-frame ROI is also defined for measuring the total active pixel area in the image.

The program generates a synthetic image containing Gaussian blobs and saves it as a grayscale BMP file. Then, a temporary reference image is cleared and a single isolated Gaussian blob is drawn for calibration purposes. This reference image is converted into a binary image using thresholding, and the number of white pixels inside a selected ROI is counted to represent the area of one object.

Next, the synthetic image containing multiple blobs is also converted into a binary image and saved as a BMP file. The total number of white pixels is calculated across the entire image. Finally, the estimated number of objects is obtained by dividing the total white-pixel area by the reference area of a single blob and rounding the result to the nearest integer.

Code of the “main” function Task 2: Real-World Image Processing

```
264     // ========================================
265     // TASK 2: Real Image (Photo)
266     // ========================================
267     static unsigned char ColorImage[NUMBER_OF_ROWS][NUMBER_OF_COLUMNS][NUMBER_OF_COLORS];
268
269     LoadBgrImageFromTrueColorBmpFile(ColorImage, (char*)"oranges.bmp");
270     ConvertColorImageToGrayImage(ColorImage, Image242);
271     StoreGrayImageAsGrayBmpFile(Image242, (char*)"Image242.bmp");
272     // Turn the grayscale image into a Binary image
273     DoThresholdUsingLUT(Image242, 150, 1);
274     StoreGrayImageAsGrayBmpFile(Image242, (char*)"Image242_Binary.bmp");
275
276     //Calibration
277     SROI refRoiReal;
278     refRoiReal.Top = 269; // real Top Y coordinate from BMPViewer
279     refRoiReal.Bottom = 395; //real Bottom Y coordinate from BMPViewer
280     refRoiReal.Left = 273; // real Left X coordinate from BMPViewer
281     refRoiReal.Right = 394; //real Right X coordinate from BMPViewer
282
283     // Count the white pixels inside that specific calibrated box (Area of ONE orange)
284     int refAreaReal = CountPixelsInROI(Image242, refRoiReal);
285     int totalAreaReal = CountPixelsInROI(Image242, fullRoi);
286
287     //Calculate the estimation: Total White Area / Single Orange Area
288     if (refAreaReal > 0)
289     {
290         float exactCount = (float)totalAreaReal / refAreaReal;
291         int roundedCount = (int)(exactCount + 0.5); // Rounding to nearest whole orange
292
293         printf("Ref Area (One Real Orange): %d pixels\n", refAreaReal);
294         printf("Total Area (All Oranges):   %d pixels\n", totalAreaReal);
295         printf("Estimated Oranges in Photo: %.2f -> Rounded: %d\n", exactCount, roundedCount);
296     }
297     else
298     {
299         printf("Error: Calibration ROI is empty. Check your coordinates!\n");
300     }
301     WaitForUserPressKey();
302 }
```

The second stage of the program processes a real-world image containing oranges. First, a color BMP image is loaded into memory and converted from a BGR color image into a grayscale image. The grayscale result is then saved for verification purposes.

Next, the grayscale image is converted into a binary image using thresholding with a threshold value of 150. This operation separates the bright orange objects from the darker background, producing a black-and-white representation suitable for area analysis. The binary image is also saved as a BMP file.

For calibration, a Region of Interest (ROI) is manually defined around a single orange using coordinates obtained from the BMP viewer. The number of white pixels inside this ROI represents the area of one reference orange. The program then calculates the total number of white pixels across the entire binary image.

Finally, the estimated number of oranges in the image is computed by dividing the total white-pixel area by the calibrated area of a single orange. The result is rounded to the nearest integer to obtain the final object count. If the calibration ROI contains no white pixels, an error message is displayed to indicate invalid calibration coordinates.

What did we learn?

- Pointer Arithmetic: Gaining proficiency in direct memory access and pointer increment operations (ptr++) for efficient low-level image processing.
- Object Counting Algorithm: Developing an area-ratio approach (Total Pixels / Reference Object Area) for accurate quantitative object estimation.
- Importance of Calibration: Understanding the significance of defining a reference object (ROI) to ensure reliable adaptation between synthetic environments and real-world scenarios.
- Performance Optimization: Utilizing Look-Up Tables (LUTs) to reduce redundant computations and applying ROI-based scanning to minimize processing overhead and improve efficiency.